

Scaling & Capacity Deep Dive

Sharding by Tenant and Prefix Key Space

Purpose: Isolate tenants for security and performance, and distribute load evenly.

Approach: Partition indices first by tenantId, then by prefix key space ranges.

Outcome: Prevents noisy neighbor issues, supports horizontal scaling by adding shards.

Benefit: Enables predictable query latency and easier scaling as tenants grow.

Adaptive Caching for Hot Prefixes

Purpose: Reduce load on backend by caching popular prefixes.

Approach: Edge nodes/CDN hold results for top queries with short TTLs.

Adaptive logic: Promote queries with high frequency; evict cold ones.

Benefit: Serves 90% of traffic directly from edge cache, cutting latency and cost.

Global QPS: 1M Queries per Second

Design target: handle 1M QPS across regions.

Distribution: 90% handled by edge caches; ~100K QPS reach backend.

Backend scaling: 50–100 query service pods per region with autoscaling.

Network: Multi-region deployment with Anycast DNS and load balancers.

Updates: 50K/sec (burst 200K)

Requirement: support high-velocity ingestion of entity updates.

Normal rate: 50K updates per second globally.

Burst handling: up to 200K/sec with stream partitioning and backpressure.

Mechanism: Kafka/Kinesis ingestion, delta index updates, background compaction.

Outcome: Fresh indices within seconds, critical for trending content.

Cost per Region

Estimate: tens to hundreds of thousands USD monthly depending on QPS and cache hit rate.

Drivers: compute for query nodes, ANN indices, and ingestion; Redis/edge caches; storage for logs and snapshots.

Optimization: higher cache hit rates reduce backend compute cost; compression and tiered storage lower storage bills.

Trade-offs: balancing latency, freshness, and cost efficiency.